

Подготовка к экзамену по C++ (ИТМО)

Тема: Move-семантика и Категории выражений

Hardcore Theory Edition

20 июня 2026 г.

1 Категории выражений (Value Categories)

В старом Си всё было просто: **lvalue** — то, что может стоять слева от знака `=` (имеет адрес в памяти), а **rvalue** — то, что справа (временные значения, числа). В современном C++ иерархия усложнилась, так как появилась Move-семантика.

1.1 Фундаментальные категории

1. **lvalue (Locator Value)**: Объект, у которого есть ИМЯ и АДРЕС в памяти. Вы можете взять его адрес оператором `&`. *Примеры: переменная `x`, разыменованный указатель `*ptr`, возвращаемое значение функции по ссылке `T&`.*
2. **prvalue (Pure rvalue)**: Чистые временные значения. У них нет имени и нет адреса. Они существуют только для того, чтобы инициализировать кого-то другого. *Примеры: число `42`, результат сложения `a + b`, возврат из функции по значению `T func()`.*
3. **xvalue (eXpiring value)**: "Умиращее значение". Это **lvalue**, от которого программист официально разрешил отпилить куски (забрать ресурсы), потому что оно всё равно скоро умрет. *Пример: результат вызова `std::move(x)`.*

Две комбинированные категории (используются в стандарте):

- **glvalue (Generalized lvalue)** = **lvalue** + **xvalue**. (Всё, что имеет адрес).
- **rvalue** = **prvalue** + **xvalue**. (Всё, из чего можно "украсть" ресурсы).

Суть на пальцах (Жизненная аналогия)

lvalue — это ваш дом. У него есть почтовый адрес, он никуда не денется.
prvalue — это летящий в окно кирпич. У него нет адреса, он существует в полете. Вы можете поймать его в руки (сохранить в переменную).
xvalue — это дом, который завтра сносят экскаватором. У него всё ещё есть адрес, но по документам вы уже имеете право выломать из него пластиковые окна и забрать себе (`std::move`).

2 Rvalue-ссылки (&&)

Обычная ссылка `T&` может указывать только на `lvalue`. Константная ссылка `const T&` может указывать на всё (и на `lvalue`, и на временные `rvalue`). **Rvalue-ссылка** `T&&` — была придумана специально, чтобы указывать **ТОЛЬКО** на `rvalue` (`rvalue` и `xvalue`).

Компилятор использует это для перегрузки функций. Если вы передаете временный объект (например, `std::string("Hi")`), компилятор предпочтет вызвать версию функции с аргументом `T&&`, а не `const T&`.

3 Иллюзия `std::move`

Главное правило: Функция `std::move` вообще ничего не перемещает! Под капотом она даже не генерирует ассемблерный код. Вся суть `std::move(x)` — это просто жесткий каст типа (преобразование):

```
static_cast<typename std::remove_reference<T>::type&&>(x)
```

Она просто берет `lvalue` (живой дом) и вешает на него бумажку `xvalue` (разрешено к сносу). Само физическое перемещение байтов памяти (воровство указателей) делает **Move-конструктор** или **Move-оператор присваивания** того класса, в который мы передаем результат.

4 Оптимизация возвращаемого значения (RVO / NRVO)

Если функция возвращает объект по значению:

```
1 std::string make_string() {  
2     std::string s = "Hello";  
3     return s;  
4 }
```

Вам **НЕ НУЖНО** писать `return std::move(s);`. Компиляторы делают *Named Return Value Optimization (NRVO)*. Они настолько умные, что конструируют объект `s` сразу в памяти той переменной, которая ожидает результат снаружи функции. Писать `std::move` при `return` локальной переменной — это антипаттерн, который отключает RVO и замедляет код!